

Window Functions

Bases de Datos — IIC2413

Hasta ahora en SQL

```
SELECT <atributos>  
FROM <relaciones>  
WHERE <condiciones>  
GROUP BY <atributos>  
HAVING <condición de agregación>
```

Agregación con `GROUP BY` **colapsa** las filas de cada grupo en una sola fila.

Hasta ahora en SQL

```
SELECT <atributos>  
FROM <relaciones>  
WHERE <condiciones>  
GROUP BY <atributos>  
HAVING <condición de agregación>
```

Agregación con `GROUP BY` **colapsa** las filas de cada grupo en una sola fila.

Pero hay ocasiones en donde queremos agrupar, sin colapsar.

Ejemplo

Una cadena de cafes registra ventas mensuales: `Ventas(sucursal, mes, monto)`

sucursal	mes	monto
Providencia	1	500000
Providencia	2	620000
Providencia	3	580000
Ñuñoa	1	310000
Ñuñoa	2	450000
Ñuñoa	3	420000
Las Condes	1	700000
Las Condes	2	680000
Las Condes	3	750000

Lo que sabemos hacer

Total de ventas por sucursal:

```
SELECT sucursal, SUM(monto) AS total  
FROM Ventas  
GROUP BY sucursal
```

sucursal	total
Las Condes	2130000
Providencia	1700000
Ñuñoa	1180000

Funciona, pero **perdemos** el detalle mes a mes.

Lo que no podemos hacer

¿Y si queremos ver, **en cada fila**, el monto del mes **junto con** el total de su sucursal?

```
SELECT sucursal, mes, monto, SUM(monto)
FROM Ventas
GROUP BY sucursal
```

Error. `mes` y `monto` no están en el `GROUP BY` ni dentro de una función de agregación.

No podemos hacer esto con los operadores que tenemos.

La solución: Window Functions

Window Functions

```
SELECT sucursal, mes, monto,  
       SUM(monto) OVER (PARTITION BY sucursal) AS total_sucursal  
FROM Ventas
```

sucursal	mes	monto	total_sucursal
Providencia	1	500000	1700000
Providencia	2	620000	1700000
Providencia	3	580000	1700000
Ñuñoa	1	310000	1180000
Ñuñoa	2	450000	1180000
Ñuñoa	3	420000	1180000
Las Condes	1	700000	2130000
Las Condes	2	680000	2130000
Las Condes	3	750000	2130000

¿Dónde se usa esto en la práctica?

Cada vez que necesitamos **comparar una fila con su contexto**, estamos en territorio de window functions:

- **Finanzas** — precio de cierre vs. promedio móvil de 30 días; ranking de acciones por retorno dentro de un sector.
- **E-commerce** — ventas acumuladas por tienda; comparar cada pedido con el promedio de su categoría.
- **Salud** — evolución de un indicador clínico paciente a paciente, mes a mes.
- **Deportes** — ranking de jugadores por puntaje dentro de su equipo; racha de victorias.
- **Redes sociales** — crecimiento de seguidores día a día; el post con más engagement por usuario.

¿Por qué hacerlo en SQL?

"Puedo exportar los datos y hacer esto en Excel / Python / R..." Sí, pero:

1. **Volumen.** Millones de filas no caben en un spreadsheet. SQL opera directo sobre la BD.
2. **Eficiencia.** El motor optimiza la ejecución (índices, paralelismo).
3. **Reproducibilidad.** Una consulta SQL es declarativa y se reutiliza. Un spreadsheet con fórmulas copiadas a mano es frágil.
4. **Composición.** El resultado es una tabla — se puede seguir consultando, filtrando, uniendo.
5. **Cercanía al dato.** Cada paso de exportación es un riesgo de inconsistencia.

Estándar SQL desde **SQL:2003**. Soportado en PostgreSQL, SQLite 3.25+ y muchos otros sistemas.

¿Qué está pasando formalmente?

La idea

Sea una relación R con esquema $(A_1, \dots, A_n)$ y una tupla $t \in R$.

Una **window function** define, para cada tupla t , un multiconjunto $W(t) \subseteq R$ llamado la **ventana** de t , y aplica una función f sobre ese multiconjunto.

El resultado extiende cada tupla con el valor calculado:

$$R' = \{ (t.A_1, \dots, t.A_n, f(W(t))) \mid t \in R \}$$

La cardinalidad (cantidad de tuplas) no cambia: $|R'| = |R|$.

Construcción de la ventana $W(t)$

1. Partición — PARTITION BY $B_1, \dots, B_k$

$$P(t) = \{s \in R \mid s.B_1 = t.B_1 \wedge \dots \wedge s.B_k = t.B_k\}$$

Construcción de la ventana $W(t)$

1. Partición — `PARTITION BY` $B_1, \dots, B_k$

$$P(t) = \{s \in R \mid s.B_1 = t.B_1 \wedge \dots \wedge s.B_k = t.B_k\}$$

2. Orden — `ORDER BY` C define un orden total $\leq_C$ sobre $P(t)$.

Construcción de la ventana $W(t)$

1. **Partición** — `PARTITION BY` $B_1, \dots, B_k$

$$P(t) = \{s \in R \mid s.B_1 = t.B_1 \wedge \dots \wedge s.B_k = t.B_k\}$$

2. **Orden** — `ORDER BY` C define un orden total $\leq_C$ sobre $P(t)$.

3. **Frame** — subconjunto de $P(t)$ relativo a la posición de t en $\leq_C$.

$$W(t) = \{s \in P(t) \mid s \text{ está en el rango del frame respecto a } t\}$$

Si no hay `PARTITION BY`: $P(t) = R$. Si no hay frame: $W(t) = P(t)$ completo.

GROUP BY vs. Window Functions

	GROUP BY	Window function
Entrada	R	R
Salida	Una fila por grupo	Una fila por tupla original
Operación	$\pi_{B, f(A)}(\gamma_B(R))$	Extiende cada t con $f(W(t))$
Cardinalidad	$ R' \leq R $	$ R' = R $

GROUP BY es una **reducción**: colapsa tuplas.

Window functions son una **extensión**: agregan información sin perder tuplas.

Sintaxis y funciones

Anatomía de una Window Function

```
función(...) OVER (  
    PARTITION BY <atributos>    -- define los grupos (ventanas)  
    ORDER BY <atributos>        -- ordena dentro de cada ventana  
    ROWS/RANGE <frame>         -- define el marco de filas  
)
```

- **PARTITION BY** — como un **GROUP BY**, pero sin colapsar filas.
- **ORDER BY** — ordena las filas dentro de cada partición.
- **Frame** — opcionalmente limita qué filas de la ventana participan.

Los tres son opcionales. **OVER ()** sin nada usa **toda la tabla** como ventana.

OVER sin PARTITION BY

Si omitimos `PARTITION BY`, la ventana es toda la tabla:

```
SELECT sucursal, mes, monto,  
       SUM(monto) OVER () AS total_global  
FROM Ventas
```

sucursal	mes	monto	total_global
Providencia	1	500000	5010000
Providencia	2	620000	5010000
...	...	...	5010000

Cada fila ve el total de **todas** las ventas.

Agregados con OVER

Funciones de agregación como Window Functions

Todas las funciones que ya conocen funcionan con `OVER` :

```
SELECT sucursal, mes, monto,  
       SUM(monto) OVER (PARTITION BY sucursal) AS total,  
       AVG(monto) OVER (PARTITION BY sucursal) AS promedio,  
       MIN(monto) OVER (PARTITION BY sucursal) AS minimo,  
       MAX(monto) OVER (PARTITION BY sucursal) AS maximo,  
       COUNT(*) OVER (PARTITION BY sucursal) AS num_meses  
FROM Ventas
```

¿Cuál es el promedio de Ñuñoa? ¿Y el máximo de Las Condes?

Porcentaje del total

Un patrón muy útil — qué **porcentaje** representa cada fila respecto a su grupo:

```
SELECT sucursal, mes, monto,  
       ROUND(100.0 * monto / SUM(monto) OVER (PARTITION BY sucursal), 1)  
       AS pct_sucursal  
FROM Ventas
```

sucursal	mes	monto	pct_sucursal
Providencia	1	500000	29.4
Providencia	2	620000	36.5
Providencia	3	580000	34.1
Nuñoa	1	310000	26.3
...	...	...	...

Funciones de ranking

ROW_NUMBER

Asigna un número secuencial a cada fila dentro de la partición:

```
SELECT sucursal, mes, monto,  
       ROW_NUMBER() OVER (PARTITION BY sucursal ORDER BY monto DESC) AS fila  
FROM Ventas
```

sucursal	mes	monto	fila
Providencia	2	620000	1
Providencia	3	580000	2
Providencia	1	500000	3
Ñuñoa	2	450000	1
Ñuñoa	3	420000	2
Ñuñoa	1	310000	3

Top-N por grupo

El mejor mes de cada sucursal:

```
SELECT *  
FROM (  
    SELECT sucursal, mes, monto,  
           ROW_NUMBER() OVER (PARTITION BY sucursal ORDER BY monto DESC) AS rn  
    FROM Ventas  
  ) sub  
WHERE rn = 1
```

sucursal	mes	monto	rn
Las Condes	3	750000	1
Nuñoa	2	450000	1
Providencia	2	620000	1

¿Cómo harían esto sin window functions?

RANK y DENSE_RANK

¿Qué pasa cuando hay **empates**?

sucursal	mes	monto
Providencia	1	500000
Providencia	2	620000
Providencia	3	620000

```
SELECT mes, monto,  
       ROW_NUMBER() OVER (ORDER BY monto DESC) AS row_num,  
       RANK()       OVER (ORDER BY monto DESC) AS rank,  
       DENSE_RANK() OVER (ORDER BY monto DESC) AS dense_rank  
FROM Ventas  
WHERE sucursal = 'Providencia'
```

RANK y DENSE_RANK — Resultado

mes	monto	row_num	rank	dense_rank
2	620000	1	1	1
3	620000	2	1	1
1	500000	3	3	2

- **ROW_NUMBER** — siempre distinto (arbitrario en empates).
- **RANK** — mismo valor en empates, **salta** posiciones (1, 1, 3).
- **DENSE_RANK** — mismo valor en empates, **no salta** (1, 1, 2).

¿Cuál usar? Depende del contexto. Podio: **RANK** . Lista compacta: **DENSE_RANK** .

NTILE

Divide las filas en **N grupos** aproximadamente iguales:

```
SELECT sucursal, mes, monto,  
       NTILE(3) OVER (ORDER BY monto DESC) AS tercil  
FROM Ventas
```

sucursal	mes	monto	tercil
Las Condes	3	750000	1
Las Condes	1	700000	1
Las Condes	2	680000	1
Providencia	2	620000	2
Providencia	3	580000	2
Providencia	1	500000	2
Ñuñoa	2	450000	3

Funciones de navegación

LAG y LEAD

Acceden a filas **anteriores** o **siguientes** dentro de la ventana:

```
SELECT sucursal, mes, monto,  
       LAG(monto) OVER (PARTITION BY sucursal ORDER BY mes) AS mes_anterior,  
       LEAD(monto) OVER (PARTITION BY sucursal ORDER BY mes) AS mes_siguiente  
FROM Ventas
```

sucursal	mes	monto	mes_anterior	mes_siguiente
Providencia	1	500000	NULL	620000
Providencia	2	620000	500000	580000
Providencia	3	580000	620000	NULL

¿Por qué **NULL** en la primera y última fila de cada partición?

Variación mensual

Calcular el **cambio** respecto al mes anterior:

```
SELECT sucursal, mes, monto,  
       monto - LAG(monto) OVER (PARTITION BY sucursal ORDER BY mes)  
       AS variacion  
FROM Ventas
```

sucursal	mes	monto	variacion
Providencia	1	500000	NULL
Providencia	2	620000	120000
Providencia	3	580000	-40000
Nuñoa	1	310000	NULL
Nuñoa	2	450000	140000
Nuñoa	3	420000	-30000

Acumulados y frames

Agregados acumulados

Cuando combinamos `OVER` con `ORDER BY`, el agregado se calcula **acumuladamente**:

```
SELECT sucursal, mes, monto,  
       SUM(monto) OVER (PARTITION BY sucursal ORDER BY mes) AS acumulado  
FROM Ventas
```

sucursal	mes	monto	acumulado
Providencia	1	500000	500000
Providencia	2	620000	1120000
Providencia	3	580000	1700000
Ñuñoa	1	310000	310000
Ñuñoa	2	450000	760000
Ñuñoa	3	420000	1180000

Frame: controlando el rango de filas

El **frame** define exactamente qué filas participan en el cálculo:

```
SUM(monto) OVER (  
  PARTITION BY sucursal  
  ORDER BY mes  
  ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING  
)
```

Suma del mes anterior + actual + siguiente.

Frame por defecto cuando hay `ORDER BY` :

```
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

Por eso `SUM(...) OVER (ORDER BY ...)` da un acumulado.

Frame — Opciones

ROWS BETWEEN <inicio> AND <fin>

Opción	Significado
UNBOUNDED PRECEDING	Desde la primera fila de la partición
N PRECEDING	N filas antes de la actual
CURRENT ROW	La fila actual
N FOLLOWING	N filas después de la actual
UNBOUNDED FOLLOWING	Hasta la última fila de la partición

Frame — Ejemplo visual

ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

mes 1: 500000	←	ventana [500000, 620000]	→ suma = 1120000
mes 2: 620000	←	ventana [500000, 620000, 580000]	→ suma = 1700000
mes 3: 580000	←	ventana [620000, 580000]	→ suma = 1200000

```
SELECT sucursal, mes, monto,  
       AVG(monto) OVER (  
         PARTITION BY sucursal ORDER BY mes  
         ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING  
       ) AS promedio_movil  
FROM Ventas
```

ROWS vs. RANGE

- **ROWS** — cuenta filas físicas.
- **RANGE** — agrupa por **valor**. Si dos filas comparten el valor en **ORDER BY**, ambas se incluyen.

```
-- ROWS: exactamente la fila anterior
```

```
SUM(monto) OVER (ORDER BY mes ROWS BETWEEN 1 PRECEDING AND CURRENT ROW)
```

```
-- RANGE: todas las filas con valor de mes <= actual
```

```
SUM(monto) OVER (ORDER BY mes RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
```

En la mayoría de los casos, **ROWS** es lo que quieren.

Juntando todo

Orden de evaluación en SQL

¿Dónde se evalúan las window functions?

1. FROM / JOIN
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT ← window functions se evalúan aquí
6. DISTINCT
7. ORDER BY
8. LIMIT / OFFSET

Por eso **no se pueden usar** en `WHERE` ni en `HAVING` .

Para filtrar por el resultado de una window function → **subconsulta**.

Resumen de funciones

Función	¿Qué hace?	¿ORDER BY?
SUM , AVG , MIN , MAX , COUNT	Agregación sobre la ventana	Opcional (acumulado si se usa)
ROW_NUMBER()	Número secuencial único	Sí
RANK()	Ranking con saltos en empates	Sí
DENSE_RANK()	Ranking sin saltos	Sí
NTILE(n)	Divide en n grupos	Sí
LAG(expr, n, default)	Valor de n filas atrás	Sí
LEAD(expr, n, default)	Valor de n filas adelante	Sí

Ejercicios

Ejercicio 1

Usando `Ventas(sucursal, mes, monto)` :

Obtener las sucursales cuyo monto en algún mes supera el promedio de esa sucursal.

Hint: subconsulta + `AVG(...) OVER (...)` .

Ejercicio 1 — Solución

```
SELECT DISTINCT sucursal
FROM (
    SELECT sucursal, monto,
           AVG(monto) OVER (PARTITION BY sucursal) AS promedio
    FROM Ventas
) sub
WHERE monto > promedio
```

Ejercicio 2

Para cada sucursal, mostrar el mes con mayor crecimiento respecto al mes anterior.

Ejercicio 2 — Solución

```
SELECT sucursal, mes, crecimiento
FROM (
    SELECT sucursal, mes, monto,
           monto - LAG(monto) OVER (PARTITION BY sucursal ORDER BY mes)
           AS crecimiento,
           ROW_NUMBER() OVER (
               PARTITION BY sucursal
               ORDER BY (monto - LAG(monto)
                       OVER (PARTITION BY sucursal ORDER BY mes)) DESC NULLS LAST
           ) AS rn
    FROM Ventas
) sub
WHERE rn = 1
```

NULLS LAST evita que el NULL del primer mes gane el ranking.

Para recordar

1. Las window functions **no colapsan filas** — esa es su gracia.
2. `PARTITION BY` define los grupos; `ORDER BY` el orden dentro del grupo.
3. `ROW_NUMBER`, `RANK`, `DENSE_RANK` → **rankings** y **top-N por grupo**.
4. `LAG` y `LEAD` → comparar con filas **vecinas** (variaciones).
5. Agregados + `OVER (ORDER BY ...)` → **acumulados**.
6. El **frame** (`ROWS BETWEEN ...`) controla qué filas se incluyen.
7. Se evalúan **después** del `WHERE` — para filtrar, usar subconsulta.